SMART-ASHA ENGINEERING DOSSIER • PART 05

Extracting Maximum Performance from Every Sensor

Sensor-by-sensor silicon audit: Current firmware utilization vs. untapped hardware capabilities

TARGET FIRMWARE	HOST CONTROLLER
`Arduino/final_eval/final_eval.ino`	`7Semi ESP32-S3 EC200U 4G LTE Smart Modem`
MULTIPLEXER	DOMAIN
`TCA9548A Dual I2C Channel Router`	`Biomedical Signal Acquisition & Edge ML`

Table of Contents

1. MLX90640 — Thermal Vision Camera
2. AS7343 — 14-Channel Spectral Sensor
3. MAX30105 — Triple-LED Optical Sensor
4. AD8232 — ECG Analog Front-End
5. MQ-135 — Gas / Respiratory Sensor
6. MPU6050 — 6-Axis IMU
7. ESP32-S3 + Quectel EC200U — The Brain & 4G/GPS Smart Modem
8. SH1106 OLED — The Display
9. System-Level Unlocks

1. MLX90640 — Thermal Vision Camera (TCA Port 3)

What You Use Today

```
mlx.getFrame(thermalFrame);          // Read 768 pixels
float mt = -99;
for(int i=0; i<768; i++)
    if(thermalFrame[i] > mt) mt = thermalFrame[i];  // Peak extraction
currentTemp = mt + boxOffset;        // Static -9.8 offset
```

You read one full frame, find the single hottest pixel, add a static offset, and feed the raw frame to the CNN.

What's Left on the Table

a) Ambient Substrate Temperature (`getTa()`)

The MLX90640 has an **internal temperature sensor** that measures the chip's own silicon temperature (T_a). This is free data — it's already computed internally during `getFrame()`. Right now you ignore it entirely.

Why it matters: The sensor output follows $V_{IR} \propto T_{obj}^4 - T_{sub}^4$. When T_{sub} rises (Delhi summer, ESP32 self-heating), the gradient shrinks and readings drift. The static `boxOffset = -9.8` cannot compensate for this.

```
// UNLOCK: Dynamic thermal offset
float Ta = mlx.getTa();   // Sensor substrate temp — already free from getFrame()
float dynamicOffset = -0.35f * (Ta - 25.0f) - 7.5f;  // Linear regression from your calibration data
currentTemp = maxPixel + dynamicOffset;
```

b) Reflected Temperature Parameter (`tr`)

In `MLX90640_CalculateTo()`, there's a parameter for **reflected ambient temperature** (T_r) — radiation bouncing off walls, ceilings, and the enclosure back into the lens. Clinical fever screening in enclosed spaces should set $T_r = T_a - 8.0$ to account for this. Currently your code uses default values and never touches this.

c) Emissivity Tuning

Human skin emissivity is $\varepsilon \approx 0.98$, but the default library setting may be lower. Setting it explicitly sharpens accuracy:

```
mlx.setEmissivity(0.98);  // Human skin
```

d) Chess Mode vs. Interleaved Mode

The MLX90640 supports two subpage refresh patterns: - **Chess Mode (default):** Checkerboard pattern — better noise performance - **Interleaved Mode:** Row-by-row — useful for motion capture

Your sensor is factory-calibrated in Chess Mode. **Keep it.** But be aware that switching to interleaved mode and reading at 4 Hz (instead of the current ~2 Hz effective rate) could improve motion-tolerant fever detection.

e) Region-of-Interest (ROI) Forehead Extraction

Instead of finding the single hottest pixel across all 768, you can define a **forehead ROI** in the center of the frame (rows 4–12, columns 8–24) to exclude peripheral hot objects (lamps, hands, cup of tea):

```
// ROI: Central 16x8 block (rows 4-11, cols 8-23)
float maxForehead = -99;
for (int row = 4; row < 12; row++) {
    for (int col = 8; col < 24; col++) {
        int idx = row * 32 + col;
        if (thermalFrame[idx] > maxForehead) maxForehead = thermalFrame[idx];
    }
}
```

f) Multi-Frame Temporal Averaging

Instead of trusting a single noisy frame, average the peak temperature across 3–5 consecutive frames:

```
static float tempHistory[5];
static int tempIdx = 0;
tempHistory[tempIdx++ % 5] = maxForehead + dynamicOffset;
// Use median or mean of tempHistory for final temperature
```

2. AS7343 — 14-Channel Spectral Sensor (TCA Port 4)

What You Use Today

```
as7343.readAllChannels(specReadings);   // Read all 18 raw counts
f5Sum += specReadings[5];                // Only F5 (550 nm) used
// ...
String anemia = (avgF5 > 1500) ? "RISK" : "NORMAL";  // Single absolute threshold
```

You read **all 14 spectral channels + 4 auxiliary channels** but only use **one** (F5 at 550 nm). That's using 7% of the sensor's capability.

What's Left on the Table

a) Full Channel Wavelength Map (All Already Being Read!)

Array Index	Channel	Wavelength	Clinical Relevance
0	F1	405 nm (Violet)	**Bilirubin absorption** — Jaundice screening
1	F2	425 nm (Violet-Blue)	**Bilirubin peak absorption** (450 nm vicinity)
2	FZ	450 nm (Blue)	**Peak bilirubin absorption** — neonatal jaundice gold standard wavelength
3	F3	475 nm (Blue-Cyan)	Melanin baseline reference
4	F4	515 nm (Green)	**Deoxyhemoglobin absorption shoulder**
5	**F5**	**550 nm (Green-Yellow)**	**Oxyhemoglobin absorption peak** — currently used
6	FY	555 nm (Yellow-Green)	Secondary hemoglobin reference
7	FXL	600 nm (Orange)	**Methemoglobin detection** window
8	F6	640 nm (Red)	Deep-red hemoglobin transition
9	F7	690 nm (Deep Red)	Deoxyhemoglobin/oxyhemoglobin crossover region
10	F8	745 nm (Near-IR)	**Isobestic reference** — hemoglobin-independent
11	NIR	855 nm (Near-IR)	**Tissue penetration reference** — water/lipid window
12	Clear	Broadband	Ambient light baseline
13	Flicker	50/60 Hz	**Ambient light noise detection**

CRITICAL INSIGHT

You are already paying the I2C time cost to read all 18 values. Using more channels costs **zero additional time or power**.

b) Skin-Independent Anemia Ratio (F5 / F8)

The Beer-Lambert Law states that the ratio of two wavelengths cancels out optical path length and melanin absorption:

$$\text{Hemoglobin Index} = \frac{F5\ (550\ \text{nm})}{F8\ (745\ \text{nm}) + 1}$$

```
float avgF5 = (specSamples > 0) ? (float)f5Sum / specSamples : 0;
float avgF8 = (specSamples > 0) ? (float)f8Sum / specSamples : 1.0f;
float hbRatio = avgF5 / (avgF8 + 0.01f);

const char* anemia = (hbRatio > 1.35f) ? "RISK" : "NORMAL";
```

c) Jaundice Screening (F1/FZ Bilirubin Index)

Bilirubin absorbs strongly at 450 nm (FZ channel). You can add a **free jaundice screening** to every scan:

$$\text{Bilirubin Index} = \frac{FZ\ (450\ \text{nm})}{F8\ (745\ \text{nm}) + 1}$$

```
// Accumulate during SCANNING:
fzSum += specReadings[2];  // FZ channel = index 2

// In RESULTS:
float avgFZ = (specSamples > 0) ? (float)fzSum / specSamples : 0;
float biliIndex = avgFZ / (avgF8 + 0.01f);
const char* jaundice = (biliIndex > 1.8f) ? "RISK" : "NORMAL";
```

> 💡 **PRO-TIP**
>
> This is especially relevant for neonatal care — ASHA workers screen newborns within 48 hours of birth. A non-invasive jaundice indicator at zero extra hardware cost is a significant clinical win.

d) Ambient Light Compensation (Clear + Flicker Channels)

Channel 12 (Clear) reads broadband ambient light. Channel 13 detects 50/60 Hz flicker. You can use these to: 1. **Reject noisy readings** when ambient light exceeds a threshold 2. **Normalize spectral channels** by dividing each by the Clear channel

```
uint16_t ambient = specReadings[12];  // Clear channel
if (ambient > 5000) {
    // Too much ambient light leakage — flag as invalid reading
    specValid = false;
} else {
    // Normalize: cancel out ambient contamination
    float normalizedF5 = (float)specReadings[5] / (ambient + 1);
}
```

e) SMUX Configuration for Faster Single-Condition Reads

The AS7343 has a programmable **Super MUX (SMUX)** that controls which photodiode channels are routed to the 6 ADCs. Instead of reading all 18 channels (3 SMUX cycles), you can configure a custom SMUX map to read **only F5, F8, and FZ** in a single cycle — cutting spectral read time by 66%.

f) Optimized Integration Time

Current settings: `ATIME=100, ASTEP=999` gives $t_{int} = (100+1)(999+1) \times 2.78\,\mu s \approx 281\text{ ms}$

That's quite long. For reflective measurements on skin with `GAIN=128X`, you can often reduce this:

```
as7343.setATIME(29);    // 30 cycles
as7343.setASTEP(599);   // 600 steps
// New t_int = 30 x 600 x 2.78 us = 50 ms  (5.6x faster)
```

---

3. MAX30105 — Triple-LED Optical Sensor (Wire1)

What You Use Today

```
pulse.setup();
pulse.setPulseAmplitudeRed(0x3F);
pulse.setPulseAmplitudeIR(0x3F);
// In loop:
long ir = pulse.getIR();
float r = (float)pulse.getRed() / (float)ir;
currentSpO2 = constrain(110-(25*r), 85, 99);
```

You use Red and IR for SpO2, and IR for beat detection. The third LED and the built-in temperature sensor are completely ignored.

What's Left on the Table

a) The Green LED (Mode 3: Multi-LED)

The MAX30105 has **three LEDs: Red (660 nm), IR (880 nm), and Green (530 nm)**. Green PPG has significantly better signal-to-noise ratio for heart rate detection than IR because green light has higher hemoglobin absorption contrast in capillary beds.

```
// In setup():
pulse.setup(0x3F, 4, 3, 100, 411, 4096);
// Arguments: LED power, sample avg, LED mode (3 = all three), sample rate, pulse width, ADC range

// In loop():
uint32_t green = pulse.getGreen();
// Use green channel for BPM — significantly cleaner PPG waveform
```

Why it matters: Your current BPM uses IR via `checkForBeat(ir)`. Green PPG is the same technology used by Apple Watch, Fitbit, and every modern wearable for heart rate — it's more reliable for finger-based PPG than IR.

b) Built-In Die Temperature Sensor

The MAX30105 has an internal temperature sensor (plus/minus 1 degree C accuracy). It's designed for LED calibration, but it gives you a **free independent temperature reference**:

```
float dieTempC = pulse.readTemperature();  // Returns die temperature in degrees C
```

You can use this as: - A **sanity check** against the MLX90640 readings - An **ambient drift monitor** (if the MAX30105 die is warming up, so is everything else)

c) Proper SpO2 Calibration Curve

Your current formula:

```
currentSpO2 = constrain(110-(25*r), 85, 99);
```

This is a linear approximation. The actual SpO2-R relationship is non-linear and sensor-specific. A better empirical curve:

```
// Quadratic calibration (derived from Maxim application note):
// Requires AC/DC separation of Red and IR signals
float R = (acRed / dcRed) / (acIR / dcIR);
float spo2 = -45.060f * R * R + 30.354f * R + 94.845f;
currentSpO2 = constrain((int)spo2, 70, 100);
```

To get `acRed/dcRed` and `acIR/dcIR`, you need to separate the AC (pulsatile) and DC (baseline) components using a simple digital high-pass filter on the raw Red and IR streams.

d) FIFO Buffer and Sample Averaging

The MAX30105 has a 32-sample deep hardware FIFO with programmable sample averaging (1x, 2x, 4x, 8x, 16x, 32x). Currently `pulse.setup()` uses defaults. Enabling 4x averaging smooths out motion artifacts:

```
pulse.setup(0x3F, 4, 2, 100, 411, 4096);
//                  ^ sampleAverage = 4  (hardware-averaged, no CPU cost)
```

e) Proximity Mode for Auto-Wake

The MAX30105 has a built-in **proximity interrupt** that can wake the sensor only when a finger is detected. Instead of polling `ir > 40000` every loop iteration, let the sensor hardware do it:

```
pulse.enablePROX();
pulse.setPROXThreshold(0x14);  // ~40k equivalent
```

4. AD8232 — ECG Analog Front-End (GPIO 4)

What You Use Today

```
pinMode(ecgPin, INPUT);
int ecgRaw = analogRead(ecgPin);
if (ecgRaw > 2300 && (millis() - lastRPeak > 250))  // Simple threshold R-peak detection
```

You read the raw analog output and apply a single threshold for R-peak detection. The AD8232's lead-off detection pins are not connected.

What's Left on the Table

a) Lead-Off Detection (LO+ / LO-)

The AD8232 has two digital output pins — **LO+** and **LO-** — that go HIGH when an electrode is disconnected. You're currently ignoring them entirely, meaning if a patient's electrode pad peels off mid-scan, the firmware happily processes noise as "ECG data."

```
// In setup():
const int loPlus = 36;   // Connect LO+ to any free GPIO
const int loMinus = 37;  // Connect LO- to any free GPIO
pinMode(loPlus, INPUT);
pinMode(loMinus, INPUT);

// In loop() before processing ECG:
bool leadsOff = (digitalRead(loPlus) == HIGH) || (digitalRead(loMinus) == HIGH);
if (leadsOff) {
    // Don't process ECG — show "ATTACH LEADS" on OLED
    return;
}
```

> ⚠️ **WARNING & SAFETY**
>
> Without lead-off detection, the device cannot distinguish between a real cardiac arrhythmia and a detached electrode. This is the most important missing safety feature.

b) Digital Bandpass Filter (Pan-Tompkins Preprocessing)

Raw `analogRead()` at 12-bit resolution includes: - 50 Hz power-line hum (India mains) - Baseline wander (breathing motion, less than 0.5 Hz) - EMG noise from muscle tremor (greater than 40 Hz)

A simple 2nd-order IIR bandpass (5-15 Hz) isolates only the QRS complex energy:

```
// Coefficients for 5-15 Hz bandpass at 200 Hz sample rate (Butterworth 2nd order)
// Pre-computed from scipy.signal.butter(2, [5, 15], btype='band', fs=200)
float b[] = {0.0675f, 0.0f, -0.1349f, 0.0f, 0.0675f};
float a[] = {1.0f, -2.7693f, 3.1044f, -1.7083f, 0.3849f};

float ecgX[5] = {0}, ecgY[5] = {0};

float filterECG(float input) {
    // Shift buffers
    for (int i = 4; i > 0; i--) { ecgX[i] = ecgX[i-1]; ecgY[i] = ecgY[i-1]; }
    ecgX[0] = input;
    ecgY[0] = 0;
    for (int i = 0; i < 5; i++) ecgY[0] += b[i] * ecgX[i];
    for (int i = 1; i < 5; i++) ecgY[0] -= a[i] * ecgY[i];
    return ecgY[0];
}
```

c) Adaptive R-Peak Threshold

Your current threshold is hardcoded at `2300`. This breaks if: - The patient has lower-amplitude R-waves (thin chest wall) - Electrode placement varies - The baseline drifts over 60 seconds

Replace with an adaptive threshold that tracks the running signal envelope:

```
static float peakEnvelope = 2000;
float filtered = filterECG((float)ecgRaw);

// Envelope decay (0.995 = ~500 ms decay at 200 Hz)
peakEnvelope = max(abs(filtered), peakEnvelope * 0.995f);

// R-peak fires at 60% of recent peak envelope
float adaptiveThreshold = peakEnvelope * 0.6f;
if (filtered > adaptiveThreshold && (millis() - lastRPeak > 250)) {
    // R-peak detected
}
```

d) ST-Segment Deviation (Basic Ischemia Flag)

After detecting the R-peak, the ST segment occurs ~80 ms later. If you're sampling at 200 Hz, that's ~16 samples after the R-peak. You can measure the voltage level at that point relative to the PR baseline to flag potential ischemia:

```
// In the timer ISR, after R-peak detection:
if (samplesSinceRPeak == 16) {  // ~80 ms after R at 200 Hz
    float stLevel = filteredECG;
    float stDeviation = abs(stLevel - prBaseline);
    if (stDeviation > stThreshold) {
        stElevationCount++;  // Flag for clinical review
    }
}
```

> 🛡️ **CLINICAL CAUTION**
>
> ST-segment analysis with a single-lead consumer ECG is for **screening/flagging only**, not diagnosis. But for rural triage where the alternative is no screening at all, it adds value.

---

5. MQ-135 — Gas / Respiratory Sensor (GPIO 6)

What You Use Today

```
gasSum += gasRaw; gasSamples++;
if(gasRaw > peakGas) peakGas = gasRaw;
// ...
String resp = (peakGas - avgGas < 300) ? "RISK" : "GOOD";
```

You read raw ADC values and check if the peak-to-average delta exceeds 300 counts.

What's Left on the Table

a) Breath Rate Detection (Counting Exhale Peaks)

Each exhale sends a burst of CO2 and VOCs past the MQ-135, creating a periodic peak in the signal. Instead of just computing peak-minus-average, you can **count the number of peaks per minute** to derive respiratory rate:

```
int breathCount = 0;
unsigned long lastBreathPeak = 0;

// In the sampling loop:
// Simple peak detector with 3-second refractory:
if (gasRaw > (avgGas + 200) && (millis() - lastBreathPeak > 3000)) {
    breathCount++;
    lastBreathPeak = millis();
}

// In RESULTS:
float breathsPerMin = breathCount * (60000.0f / (millis() - scanStartTime));
const char* resp;
if (breathsPerMin < 12 || breathsPerMin > 20) resp = "ABNORMAL";
else resp = "NORMAL";
```

Normal adult respiratory rate: 12-20 breaths/min. Below 12 = bradypnea. Above 20 = tachypnea.

b) Warm-Up Compensation

MQ-135 requires a ~24-hour burn-in for stable readings, and readings drift significantly in the first 2-3 minutes after power-on. You can add a warm-up discard period:

```
// Discard gas readings for the first 120 seconds after boot:
if (millis() > 120000) {
    gasSum += gasRaw;
    gasSamples++;
}
```

c) Resistance-to-PPM Conversion

The raw ADC reading is just a voltage divider output. Converting to approximate PPM of CO2:

```
float sensorVoltage = gasRaw * (3.3f / 4095.0f);
float sensorResistance = (3.3f - sensorVoltage) / sensorVoltage * 10000.0f;  // 10K load resistor
float ratio = sensorResistance / R0;  // R0 = resistance in clean air (calibrated once)
float ppmCO2 = 116.6020682f * pow(ratio, -2.769034857f);  // From MQ-135 datasheet curve
```

---

6. MPU6050 — 6-Axis IMU (TCA Port 2)

What You Use Today

Nothing. The MPU6050 is wired to TCA Port 2 and listed in the pinout guide, but `final_eval.ino` never calls `tcaSelect(2)` or reads any accelerometer/gyroscope data.

What's Sitting Completely Unused

a) Motion Artifact Rejection

The MPU6050 can detect when the patient is moving during the scan. If acceleration exceeds a threshold, flag ECG and SpO2 readings as unreliable:

```
#include <MPU6050_light.h>
MPU6050 mpu(Wire);

// In setup():
tcaSelect(2);
mpu.begin();
mpu.calcOffsets();  // Calibrate with device stationary

// In SCANNING, during the I2C mutex block:
tcaSelect(2);
mpu.update();
float totalAccel = sqrt(sq(mpu.getAccX()) + sq(mpu.getAccY()) + sq(mpu.getAccZ()));
bool patientMoving = (abs(totalAccel - 1.0f) > 0.15f);  // Deviation from 1g = motion
```

b) Respiratory Rate from Chest-Wall Motion

If the sensor is placed on the patient's chest (in a stethoscope sleeve), the Z-axis accelerometer captures the subtle rise/fall of breathing. This gives you a **second independent respiratory rate measurement** to cross-validate the gas sensor:

```
// Low-pass filter the Z-axis at ~0.5 Hz to isolate breathing
static float zFiltered = 0;
zFiltered = 0.05f * mpu.getAccZ() + 0.95f * zFiltered;
// Count zero-crossings of (zFiltered - baseline) for breath rate
```

c) Fall Detection / Emergency Alert

A sudden spike in acceleration (greater than 3g) followed by stillness could indicate a patient or worker fall — useful for elderly patient screening:

```
if (totalAccel > 3.0f) {
    fallDetected = true;
    // Show alert on OLED
}
```

d) Posture Detection for Calibration

Heart rate and SpO2 readings are posture-dependent (sitting vs. supine). The MPU6050 can detect if the patient is sitting upright or lying down and adjust thresholds accordingly.

7. ESP32-S3 + Quectel EC200U — The Brain Itself (7Semi Smart Modem)

What You Use Today

- Dual-core FreeRTOS (Core 0: AI, Core 1: sensors + UI)
- 12-bit ADC for ECG and gas
- I2C x 2 buses
- 16 KB stack for AI task
- **4G LTE modem (EC200U) — NOT USED**
- **GPS/GNSS receiver — NOT USED**
- **SMS capability — NOT USED**

What's Left on the Table

a) Hardware Timer Interrupts for Guaranteed Sampling

Your `analogRead()` calls are in `loop()`, which is blocked by I2C. Use ESP32 hardware timers for jitter-free sampling:

```
hw_timer_t *ecgTimer = NULL;
volatile int ecgRingBuf[256];
volatile uint8_t ecgHead = 0;

void IRAM_ATTR onEcgTimer() {
    ecgRingBuf[ecgHead++] = analogRead(ecgPin);
}

// In setup():
ecgTimer = timerBegin(0, 80, true);      // 1 us ticks
timerAttachInterrupt(ecgTimer, &onEcgTimer, true);
timerAlarmWrite(ecgTimer, 5000, true);    // 5 ms = 200 Hz
timerAlarmEnable(ecgTimer);
```

b) Deep Sleep Between Patients

Between patients (IDLE state), the ESP32 draws ~240 mA doing nothing. Use light sleep to drop to ~2 mA, waking on a touch pad or MAX30105 proximity interrupt:

```
case IDLE:
    // After showing "SMART ASHA READY" screen:
    esp_sleep_enable_timer_wakeup(1000000);  // Wake every 1 second to check
    esp_light_sleep_start();
    break;
```

c) ADC Calibration (eFuse)

The ESP32-S3 has factory-burned calibration data in eFuse that corrects ADC non-linearity. Enable it:

```
#include <esp_adc_cal.h>
esp_adc_cal_characteristics_t adc_chars;
esp_adc_cal_characterize(ADC_UNIT_1, ADC_ATTEN_DB_11, ADC_WIDTH_BIT_12, 0, &adc_chars);

// Use calibrated reads:
uint32_t voltage_mv = esp_adc_cal_raw_to_voltage(analogRead(ecgPin), &adc_chars);
```

d) PSRAM for Patient Record Buffer

You have OPI PSRAM enabled but don't use it. Allocate large buffers in PSRAM to free internal SRAM:

```
float* rawEcgLog = (float*)ps_malloc(sizeof(float) * 12000);  // 60s @ 200 Hz
// Stores the full ECG trace for post-scan review or BLE export
```

e) NVS/LittleFS for Persistent Patient Storage

Store screening results that survive power cycles:

```
#include <Preferences.h>
Preferences prefs;

struct PatientRecord {
    uint32_t timestamp;
    float temp;
    uint8_t spo2;
    uint8_t bpm;
    uint8_t arrhythmia;
    uint8_t anemia;
    uint8_t jaundice;
    uint8_t breathRate;
};  // 16 bytes per patient → 1000+ records in 16 KB

void savePatient(PatientRecord rec) {
    prefs.begin("patients", false);
    uint32_t count = prefs.getUInt("count", 0);
    String key = "p" + String(count);
    prefs.putBytes(key.c_str(), &rec, sizeof(rec));
    prefs.putUInt("count", count + 1);
    prefs.end();
}
```

f) 4G LTE Data Upload via EC200U (Quectel Modem)

The 7Semi board has a Quectel EC200U-CN cellular modem connected internally via UART. This is **completely unused** in the current firmware. With a Jio/Airtel/BSNL SIM card inserted, the device can push data over 4G without any smartphone:

```
// Initialize modem UART (check 7Semi docs for exact pins)
HardwareSerial modemSerial(1);  // UART1
modemSerial.begin(115200, SERIAL_8N1, RX_PIN, TX_PIN);

// Send AT command to transmit data via HTTP POST:
void sendToCloud(PatientRecord rec) {
    modemSerial.println("AT+HTTPINIT");
    delay(100);
    modemSerial.println("AT+HTTPPARA=\"URL\",\"https://your-dashboard.in/api/screening\"");
    delay(100);
    // ... construct JSON payload and POST ...
}
```

g) GPS/GNSS Geotagging

The EC200U includes a multi-constellation GNSS receiver (GPS, GLONASS, BeiDou, Galileo). Every screening can be automatically geotagged:

```
void getGPSLocation(float &lat, float &lon) {
    modemSerial.println("AT+QGPS=1");          // Power on GNSS
    delay(2000);
    modemSerial.println("AT+QGPSLOC=2");       // Get location (format: ddmm.mmmm)
    // Parse response for latitude and longitude
}

// In RESULTS: attach GPS coords to PatientRecord
PatientRecord rec;
getGPSLocation(rec.latitude, rec.longitude);
```

h) Emergency SMS Alert for Critical Patients

When triage score >= 50 (CRITICAL), auto-send SMS to the nearest PHC or 108 ambulance:

```
void sendEmergencySMS(float lat, float lon, const char* vitals) {
    modemSerial.println("AT+CMGF=1");          // Set SMS text mode
    delay(100);
    modemSerial.println("AT+CMGS=\"+91XXXXXXXXXX\"");  // PHC phone number
    delay(100);
    modemSerial.print("CRITICAL PATIENT - Smart ASHA Alert\n");
    modemSerial.print("Location: "); modemSerial.print(lat, 6);
    modemSerial.print(","); modemSerial.println(lon, 6);
    modemSerial.print("Vitals: "); modemSerial.println(vitals);
    modemSerial.write(0x1A);  // Ctrl+Z to send
}
```

> **CRITICAL INSIGHT**
>
> The 4G + GPS + SMS capability is a **massive untapped differentiator**. No competitor at any price provides standalone cellular connectivity built into the diagnostic device itself. Healthcube requires an external smartphone for all connectivity.

---

8. SH1106 OLED — The Display (TCA Port 7)

What You Use Today

Full-frame buffer mode (`U8G2_..._F_...`), clearing and redrawing the entire 128×64 screen every 1.5 seconds.

What's Left on the Table

a) Page Buffer Mode for Speed

Switch from full-frame buffer (`_F_`) to page buffer (`_1_` or `_2_`). This uses 1/8th the RAM and sends only changed pages:

```
U8G2_SH1106_128X64_NONAME_1_HW_I2C u8g2(U8G2_R0, U8X8_PIN_NONE);
// Draw with firstPage()/nextPage() loop instead of clearBuffer()/sendBuffer()
```

Trade-off: Drawing code is slightly more complex, but OLED refresh time drops from ~120 ms to ~15 ms per page.

b) Partial Screen Updates

Only redraw the regions that changed. The header bar (BPM, SpO2, Temp) and the ECG graph change frequently, but the footer stays static:

```
// Draw header bar only when values change
u8g2.setClipWindow(0, 0, 128, 12);
// ... draw header ...
u8g2.setMaxClipWindow();  // Reset
```

c) Contrast/Brightness Control for Outdoor Use

In bright sunlight (village outdoor screening), boost OLED contrast:

```
u8g2.setContrast(255);  // Max brightness for outdoor visibility
```

d) Screen Rotation for Enclosure Flexibility

If the OLED is mounted upside-down in the sleeve:

```
U8G2_SH1106_128X64_NONAME_F_HW_I2C u8g2(U8G2_R2, U8X8_PIN_NONE);  // 180-degree rotation
```

---

9. System-Level Unlocks

a) Cross-Sensor Fusion Confidence Score

Instead of 6 independent yes/no flags, compute a **composite triage severity score (0-100)**:

```
float severityScore = 0;
severityScore += (fProb > 60) ? 25 : 0;           // Fever: 25 pts
severityScore += (avgO2 < 94)  ? 30 : 0;          // Hypoxia: 30 pts
severityScore += (stdDev > 115) ? 20 : 0;          // Arrhythmia: 20 pts
severityScore += (hbRatio > 1.35) ? 15 : 0;       // Anemia: 15 pts
severityScore += (breathRate < 12 || breathRate > 20) ? 10 : 0;  // Resp: 10 pts

const char* triage;
if (severityScore ⩾ 50) triage = "CRITICAL - REFER NOW";
else if (severityScore ⩾ 25) triage = "WARNING - MONITOR";
else triage = "NORMAL";
```

b) BLE Data Export (Web-Bluetooth)

ESP32-S3 has built-in BLE 5.0. Add a GATT server that exposes patient records:

```
#include <BLEDevice.h>
#include <BLEServer.h>
#include <BLEUtils.h>

// Create a BLE service with a characteristic for each vital
// An ASHA worker opens Chrome → connects → downloads all records
// Zero app installation required (Web Bluetooth API)
```

c) Audio/Buzzer Alert for Critical Findings

Connect a passive buzzer to any free GPIO for an audible alert when severity is critical:

```
const int buzzerPin = 5;
if (severityScore ⩾ 50) {
    tone(buzzerPin, 2000, 500);  // 2 kHz beep for 500 ms
    delay(200);
    tone(buzzerPin, 2000, 500);  // Double beep = critical
}
```

d) Watchdog Timer for Crash Recovery

Add a hardware watchdog so if the firmware hangs (I2C lockup, stack overflow), it automatically reboots:

```
#include <esp_task_wdt.h>

// In setup():
esp_task_wdt_init(10, true);  // 10-second timeout, auto-restart on failure
esp_task_wdt_add(NULL);       // Monitor the current task

// In loop() — feed the watchdog regularly:
esp_task_wdt_reset();
```

Quick-Reference: Capability Utilization Dashboard

Sensor	Capabilities Available	Currently Used	Utilization	Top Unlock
MLX90640	6	2	33%	Dynamic `getTa()` offset, ROI extraction
AS7343	14 channels + flicker	1 channel (F5)	**7%**	F5/F8 anemia ratio, FZ jaundice, ambient rejection
MAX30105	3 LEDs + temp + FIFO + proximity	Red + IR only	40%	Green LED PPG, die temperature, proximity wake
AD8232	ECG + Lead-off + filtering	Raw ADC only	25%	Lead-off detection, bandpass filter, adaptive threshold
MQ-135	PPM conversion + breath rate	Peak-avg delta	20%	Breath rate counting, warm-up compensation
MPU6050	6-axis accel+gyro	**0% — Not used**	**0%**	Motion artifact rejection, chest-wall respiratory rate
EC200U 4G LTE	4G data + SMS + MQTT + HTTP	**0% — Not used**	**0%**	Emergency SMS alerts, cloud upload, teleconsultation
EC200U GNSS	GPS + GLONASS + BeiDou + Galileo	**0% — Not used**	**0%**	Geo-tagged patient records, disease prevalence mapping
ESP32-S3	Timers + ULP + PSRAM + NVS + BLE + ADC cal	Basic dual-core	35%	HW timer ISR, PSRAM buffers, NVS storage, BLE export
SH1106 OLED	Page mode + contrast + partial update	Full-frame redraw	30%	Page buffer mode (8x faster), outdoor contrast boost

CRITICAL INSIGHT

The EC200U (4G + GPS), AS7343, and MPU6050 represent the largest untapped potential. You have built-in cellular connectivity and GPS that no competitor at any price has as a standalone feature — and it's sitting completely unused. The AS7343 reads all 18 spectral channels but only uses 1, and the IMU is wired but has zero lines of code. These are free clinical and connectivity features waiting to be activated with pure Arduino IDE code changes.